

# Callbacks & Asynchronous Programming

How a single-threaded program does thousands of things at once — the event loop, callbacks, futures, and `async` / `await` — and why a node is built this way.

---

SUBJECT      hathor-core · Part I · Track A (programming concepts)

CHAPTER     02 · Foundations · Concepts

BRANCH      study/hathor-core-studies

EDITION     2026 · v1

---

Concurrency · Blocking I/O · Threads · Event loop · Callbacks · First-class functions · Futures/Deferreds · Coroutines · `async/await`

# Table of Contents

---

|                                                              |          |
|--------------------------------------------------------------|----------|
| <b>Chapter 2 — Callbacks &amp; Asynchronous Programming</b>  | <b>3</b> |
| 2.1 The problem: waiting is everywhere                       | 3        |
| 2.2 Three ways to do many things at once                     | 4        |
| 2.3 Callbacks — "call me back when it's ready"               | 5        |
| 2.4 The event loop — the engine that runs the callbacks      | 7        |
| 2.5 Callback hell, and the rescue: futures                   | 8        |
| 2.6 async/await — asynchronous code that reads top-to-bottom | 9        |
| 2.7 The one rule, and the escape hatch for CPU work          | 10       |
| 2.8 Bridge — asynchrony in hathor-core                       | 10       |
| Recap                                                        | 11       |

# Chapter 2 — Callbacks & Asynchronous Programming

---

## What you'll learn in this chapter

- Why a program spends most of its life waiting, and why naive blocking code wastes that time.
- The three ways to do many things at once — processes, threads, and a single-threaded **event loop** — and the trade-offs that make a node choose the last.
- **Callbacks**: handing over a function to be run later, built on the idea that functions are values.
- How an **event loop** actually works — by building a tiny one from scratch and tracing it.
- The escape from "callback hell": **futures/promises**, and then **async / await**, which lets you write asynchronous code that reads like straight-line code.
- The one unbreakable rule of this model — never block the loop — and a **bridge** to where every piece reappears in `hathor-core` (the Twisted reactor, Deferreds, the thread pool).

This is the most important concept in the book for understanding `hathor-core`. Almost every surprising thing about the codebase — why methods hand each other functions, why you must never call `time.sleep`, why there is a single "reactor" that everything runs on — follows from the ideas here. As with Chapter 1, there is no Hathor source in the body; we build the concepts on plain Python and toy examples, and the closing Bridge (§2.8) maps each one onto the real code.

A note before we start: in Chapter 1 the objects were static — data sitting still, guarded by rules. This chapter is about time: how a program decides what to do **next** when many things could happen and most of them involve waiting.

---

## 2.1 The problem: waiting is everywhere

Run a stopwatch on what a full node actually does, and you find it spends the overwhelming majority of its life **waiting** — for bytes to arrive from a peer across the internet, for the disk to return a record, for a timer to fire. Computation — verifying a signature, checking a rule — is a thin sliver between long stretches of waiting.

This split has a name. Work that is dominated by waiting for input/output is **I/O-bound**<sup>1</sup>; work dominated by actual calculation is **CPU-bound**<sup>2</sup>. A node is overwhelmingly I/O-bound. That single fact shapes its entire architecture, because how you handle waiting is the difference between a node that serves one peer and a node that serves thousands.

Here is the naive way, and why it fails. Suppose reading from a network connection looks like this:

```
data = connection.recv(1024)  # BLOCKS: the program stops here until bytes arrive
process(data)
```

The call to `recv` is **blocking**<sup>3</sup>: the line does not return until data is available, and the entire program is frozen on that line in the meantime. If the peer is slow, or silent for thirty seconds, your program does nothing for thirty seconds — it cannot read from a second peer, cannot answer a wallet, cannot fire a timer. One slow conversation stalls everything.

Blocking model, one thread, two peers:

```
recv(peerA) ——— [ waiting 30s for peerA ] ———> got data —> now recv(peerB)...
                ▲
                peerB had data ready the whole time, but we never looked.
```

For a node that must hold thousands of simultaneous connections, blocking is a non-starter. The rest of the chapter is about the ways out.

## 2.2 Three ways to do many things at once

The general problem is **concurrency**<sup>4</sup>: making progress on several tasks over the same span of time. (Note: concurrency is not the same as parallelism<sup>5</sup> — doing things at literally the same instant on multiple CPU cores. You can be concurrent on a single core by switching between tasks quickly; you are only parallel if two things truly run at once.) There are three classic approaches.

**1. Multiple processes.** Run several copies of the program, each a separate OS process with its own memory. Robust — a crash in one doesn't touch the others — but heavyweight: processes are expensive to create and can't share data easily. Good for "run N independent workers," poor for "manage 5,000 chatty connections that share state."

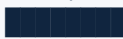
**2. Multiple threads.** A single process can have several **threads**<sup>6</sup> of execution, sharing the same memory, with the operating system rapidly switching between them (preemptive<sup>7</sup> multitasking — the OS can interrupt a thread at any moment). Threads

are lighter than processes, but sharing memory is exactly the danger: if two threads touch the same data at once, you get **race conditions**<sup>8</sup> — bugs that depend on the precise, unpredictable timing of the switches. Defending against them means **locks**<sup>9</sup>, and locks bring their own miseries (forgotten locks corrupt data; over-eager locks cause **deadlocks**<sup>10</sup> where two threads wait on each other forever). Threads are notoriously hard to get right at scale.

**Python aside — the GIL.** In the standard Python interpreter (CPython) there is a **Global Interpreter Lock**<sup>11</sup>: only one thread runs Python code at a time, even on a many-core machine. So Python threads give you concurrency but not true parallelism for Python computation. They still help for I/O-bound work (a thread releases the lock while waiting on the network), but they do not let you crunch numbers on eight cores at once. This is a major reason the Python ecosystem leans on the third approach.

**3. A single-threaded event loop.** Use exactly **one** thread, and have it switch between tasks cooperatively<sup>12</sup> — each task voluntarily yields control whenever it would otherwise wait. While task A waits for the network, the one thread goes and runs task B; when A's data finally arrives, the thread comes back to A. Because there is only one thread, **there is no shared-memory race** — two pieces of your code never run at the same instant, so you almost never need locks. This is how `hathor-core` (via Twisted) handles its thousands of connections, and it is what the rest of this chapter builds.

PARALLELISM (threads on 2 cores) – truly simultaneous

core 1:  task A   
core 2:  task B 

CONCURRENCY (one thread, event loop) – interleaved, never simultaneous

core 1:  switches whenever a task would WAIT

The trade-off of the event-loop model is the one rule we will keep returning to: since everything shares one thread, **any task that refuses to yield — that blocks, or hogs the CPU — freezes the whole program**. Cooperation is mandatory. The whole machinery below exists to make that cooperation natural to write.

## 2.3 Callbacks — "call me back when it's ready"

The event-loop model needs a way for a task to say "I'm about to wait; here's what to do when the wait is over — go run something else until then." The oldest and most fundamental way to express that is the **callback**<sup>13</sup>: a function you hand to another function, to be called back later.

Callbacks rest on one idea that trips up newcomers from some other languages: in Python, **functions are values**. A function is an ordinary object you can store in a variable, put in a list, pass as an argument, and return from another function. This property is called being **first-class**<sup>14</sup>:

```
def shout(text):
    print(text.upper())

action = shout          # store a function in a variable (no parentheses – not calling it!)
action("hi")            # HI – call it through the variable

def do_twice(fn, value): # a function that TAKES a function...
    fn(value)
    fn(value)
do_twice(shout, "echo") # ECHO / ECHO
```

A function like `do_twice` that takes or returns other functions is a **higher-order function**<sup>15</sup>. The function you pass in (`shout`, here) is, when its purpose is "run this when something happens," a **callback**.

Now the asynchronous use. Instead of a blocking read that freezes until data is ready, imagine a non-blocking read that returns immediately and promises to call your function once the data arrives:

```
def read_async(name, on_done):
    # Pretend this registers interest and returns RIGHT AWAY.
    # Some time later, when bytes are ready, the event loop will call on_done(data).
    ...

read_async("peer.dat", on_done=lambda data: print("got:", data))
print("this line runs FIRST – the read hasn't finished yet")
```

The shift in shape is the whole point. The blocking version returns the result. The asynchronous version returns nothing useful immediately and instead arranges for your callback to receive the result later. The program flows past the read without waiting, free to do other work, and the callback fires when the data is genuinely ready. The result no longer comes back to where you asked for it — it arrives in a function you left behind. That inversion is the mental adjustment async demands.

**Two senses of "callback."** Some callbacks are synchronous — `do_twice` calls `shout` immediately, right now. The interesting ones here are asynchronous — handed over now, called much later by the event loop when an event occurs. Same mechanism (a function passed as a value), different timing.

## 2.4 The event loop — the engine that runs the callbacks

We keep saying "the event loop will call your callback." What is this loop? It is less magical than it sounds — at heart it is a `while` loop over a queue of work. Let's build a miniature one and watch it run, because once you have written an event loop, you never find them mysterious again.

```
from collections import deque

class MiniLoop:
    def __init__(self):
        self._ready = deque()          # a queue of callbacks waiting to run

    def call_soon(self, callback):      # schedule a callback to run later
        self._ready.append(callback)

    def run(self):
        while self._ready:             # while there is work to do...
            callback = self._ready.popleft() # take the next ready callback
            callback()                  # ...run it to completion, then loop
```

That is a working event loop. It does one thing forever: take the next ready callback, run it, repeat. The interesting behavior appears when callbacks schedule more callbacks — that is how a task makes progress in small steps, yielding the thread between each step:

```
loop = MiniLoop()

def greet(n):
    print("hello", n)
    if n < 3:
        loop.call_soon(lambda: greet(n + 1))    # do my next step later, not now

def count(n):
    print("count", n)
    if n < 3:
        loop.call_soon(lambda: count(n + 1))

loop.call_soon(lambda: greet(1))
loop.call_soon(lambda: count(1))
loop.run()
```

Trace the output and the lesson lands:

```
hello 1      ← greet(1) runs, schedules greet(2), returns
count 1      ← count(1) runs, schedules count(2), returns
hello 2      ← greet(2) runs, schedules greet(3), returns
count 2      ← count(2) runs, schedules count(3), returns
hello 3      ← greet(3) runs (n==3, schedules nothing)
count 3      ← count(3) runs, queue empties, loop ends
```

Two independent tasks — greeting and counting — made progress **interleaved, on one thread**, with no threads and no locks. Each callback did a little work, scheduled its continuation, and returned, handing the thread back to the loop so the other task could run. A real event loop adds two things our toy lacks: **timers** (run this callback in 5 seconds — usually a list kept sorted by due-time) and **I/O readiness** (run this callback when this socket has data — the OS tells the loop which sockets are ready, via a system call like `select / epoll`). But the core — take the next ready callback, run it, repeat — is exactly what you just wrote.

Now the cardinal rule has an obvious cause. Look again at `callback()` in the loop: the loop runs **one callback at a time, to completion**, before it can touch the next. So if any callback blocks (`time.sleep(30)`, a blocking `recv`, a heavy ten-second computation), the loop is stuck inside it — every other task, every timer, every connection is frozen until that one callback returns. **Never block the loop**. Every callback must do a little and return fast.

---

## 2.5 Callback hell, and the rescue: futures

Callbacks work, but they have an ergonomics problem. Real tasks are sequences: read A, then read B, then read C, then combine them. Expressed with raw callbacks, each step nests inside the previous one's callback, and the code marches off the right edge of the screen:

```
read_async("a", lambda a:
    read_async("b", lambda b:
        read_async("c", lambda c:
            print(a, b, c)))) # the "pyramid of doom"
```

Add error handling and a loop and this becomes genuinely hard to read and reason about. The affectionate name is **callback hell**, or the pyramid of doom. Two inventions dig us out.

**The first: the future (a.k.a. promise).** Instead of passing a callback into the async function, have the async function hand you back an object that represents "a result that isn't ready yet." That object is a **future**<sup>16</sup> (Python's `asyncio` calls it `Future`; JavaScript calls it a `Promise`; Twisted, as we'll see, calls it a `Deferred`). You attach callbacks to the future — "when you eventually have the value, call this" — and, more usefully, you can chain them so each step's output feeds the next:

```
future = read_async("a") # returns a Future immediately
future.add_callback(lambda a: print("a is", a)) # run this once 'a' is ready
```



A future turns "the result arrives in a function I left behind" into "the result is an object I hold," which can be passed around, stored, and chained. It is a real improvement, but chaining `.add_callback(...).then(...)` still has visible plumbing. The second invention hides even that.

## 2.6 `async / await` — asynchronous code that reads top-to-bottom

The breakthrough is to let you write asynchronous code in the natural straight-line shape — read A, then B, then C — while it runs as cooperative callbacks under the hood. This is what `async / await` does, and it is built on a language feature called a **coroutine**<sup>17</sup>: a function that can **pause itself** partway through and be **resumed** later, right where it left off.

You define a coroutine with `async def`, and inside it you use `await` at each point where it would wait:

```
async def load_all():
    a = await read_async("a")      # pause here until 'a' is ready; let the loop run others
    b = await read_async("b")      # then pause here until 'b' is ready
    c = await read_async("c")
    print(a, b, c)                 # all three ready, in order, no nesting
```

Read `await X` as: **"pause this coroutine here, hand the one thread back to the event loop so it can run other tasks, and resume me right here with the result once X is ready."** The pyramid of §2.5 is gone; the code looks sequential and reads top-to-bottom, but at every `await` the function quietly steps aside and lets the rest of the node run.

It is essential to understand what `await` does **not** do. It does not start a new thread, and it does not run things in parallel. `load_all` is still cooperative, single-threaded code: `await` is precisely the yield point where this task volunteers to step aside. Between two `await`s, your code runs without interruption (no other task can sneak in — that's why you rarely need locks). The event loop — now usually Python's built-in `asyncio` loop, the grown-up version of our `MiniLoop` — is what drives these coroutines, resuming each one when the thing it awaited is ready.

Same three reads, three notations, identical behavior on one thread:

|              |                                |                                                         |
|--------------|--------------------------------|---------------------------------------------------------|
| blocking:    | <code>a = recv()</code>        | ← freezes the whole program while waiting               |
| callback:    | <code>recv(on_done=...)</code> | ← result arrives in a function you left behind          |
| async/await: | <code>a = await recv()</code>  | ← pauses just THIS task; loop runs others; resumes here |

So `async / await` is not a different model from callbacks — it is the same event-loop-and-callback model with a far nicer syntax. Under the hood, `await` suspends the coroutine and registers its resumption as, in effect, a callback on a future. You get the readability of straight-line code with the efficiency of cooperative concurrency.

---

## 2.7 The one rule, and the escape hatch for CPU work

Everything above reduces to a single operational rule, worth stating in bold because violating it is the most common way to break an event-loop program:

**Never block the event loop.** Inside any callback or coroutine, never do anything that waits without yielding and never do anything that hogs the CPU for long. Concretely: no `time.sleep()` (use the loop's async sleep, which yields); no blocking file or network calls (use the non-blocking/async versions); no ten-second number-crunching in one go. Each of these freezes every task, timer, and connection until it finishes, because — as §2.4 showed — the loop runs one callback at a time to completion.

But what about genuinely CPU-bound work that can't be made to yield — a heavy computation that must run start to finish? You cannot do it on the loop thread without freezing everything. The escape hatch is to push that work **off** the loop, onto a **thread pool**<sup>18</sup> or process pool: a small set of worker threads kept ready for exactly this. The loop hands the heavy job to a worker, gets a future back, and stays free to serve everyone else; when the worker finishes, the loop resumes whoever was waiting. This is the one place the event-loop model deliberately reaches for threads — for the rare CPU-bound task, fenced off from the main loop. (Foreshadowing the Bridge: a node's proof-of-work hashing is exactly such a task, and `hathor-core` runs it in a dedicated pool.)

---

## 2.8 Bridge — asynchrony in `hathor-core`

Every concept in this chapter has a direct counterpart in the codebase. `hathor-core` does not use Python's `asyncio` directly as its primary engine; it is built on **Twisted**, a mature asynchronous framework with its own names for these same ideas — but they are the same ideas. Here is the map.

### Bridge — async in the codebase (full treatment in the chapters named):

- **The event loop → the Twisted "reactor".** The single object our `MiniLoop` prototyped is, in production, the Twisted **reactor**: the one loop the whole node runs on. Starting the node ends with `reactor.run()` — handing the one thread to the loop forever. The reactor (and the option to swap in an `asyncio`-backed one) is **Chapter 16**, with the node's own wrapper in **Chapter 23**.
- **The future/promise → the Twisted "Deferred".** Twisted's name for "a result that isn't ready yet, with callbacks attached" is the **Deferred**. Everywhere the node does I/O you will see Deferreds returned and callbacks added to them — it is the §2.5 future under a different name — **Chapter 16**.
- **Callbacks everywhere.** A network connection in Twisted is an object whose methods ( `dataReceived` , `connectionMade` , `connectionLost` ) are callbacks the reactor invokes when those events occur — the §2.3 pattern as the backbone of the whole P2P layer — **Chapters 34-35**.
- **Timers.** The periodic jobs a node runs (checking sync state, refreshing peers) are scheduled on the reactor with Twisted's `LoopingCall` — the §2.4 "run this every N seconds" timer — **Chapters 29 & 34**.
- **The never-block rule, made visible.** Much of the codebase's shape — small methods that return quickly, work split into steps, the absence of `time.sleep` — is the §2.7 rule in practice. When you wonder "why is this broken into so many little callbacks?", the answer is almost always "so it never blocks the reactor."
- **The CPU escape hatch.** Proof-of-work hashing is CPU-bound, so the node runs it off the reactor in a dedicated thread pool ( `pow_thread_pool` ), exactly the §2.7 pattern — **Chapter 37**.

There is one more idea this chapter sets up without naming: once you accept that the node is a single thread dispatching callbacks, you need a clean way for parts of the system to announce events ("a new block arrived") so other parts can react — without everyone wiring callbacks to everyone else by hand. That is the **publish-subscribe** pattern, a named arrangement of callbacks, and it (with the other design patterns) is **Chapter 3**.

## Recap

| Concept   | One-line definition                              | Why it matters for a node                                  |
|-----------|--------------------------------------------------|------------------------------------------------------------|
| I/O-bound | Work dominated by waiting, not computing         | A node mostly waits; handling waiting well is everything   |
| Blocking  | A call that freezes the program until it returns | One blocking call stalls all peers — forbidden on the loop |

| Concept                    | One-line definition                                           | Why it matters for a node                                          |
|----------------------------|---------------------------------------------------------------|--------------------------------------------------------------------|
| Concurrency vs parallelism | Interleaved progress vs. literally-simultaneous               | The node is concurrent on one thread, not parallel                 |
| Threads + GIL              | OS-switched, shared memory; one Python thread at a time       | Races/locks are hard; the GIL blunts CPU parallelism               |
| Event loop                 | <code>while</code> : take next ready callback, run it, repeat | The single engine the whole node runs on (the reactor)             |
| Callback                   | A function handed over to be called on an event               | How tasks say "do this when the wait ends"                         |
| First-class function       | A function used as an ordinary value                          | The mechanism callbacks are built from                             |
| Future / Deferred          | An object for "a result not ready yet"                        | Tames nested callbacks; Twisted calls it a Deferred                |
| <code>async / await</code> | Straight-line syntax over cooperative callbacks               | Readable <code>async</code> ; <code>await</code> = the yield point |
| Never block the loop       | No sleeping/blocking/CPU-hogging in a callback                | Violating it freezes the entire node                               |
| Thread pool                | Workers for CPU-bound jobs, off the loop                      | Where proof-of-work hashing runs                                   |

Asynchronous programming is the answer to a node's defining condition: it spends its life waiting on a thousand slow conversations at once, and it cannot afford to freeze on any of them. The solution is one thread running an event loop, tasks that cooperate by yielding whenever they would wait, and callbacks (dressed up, at their best, as `async / await`) to resume each task when its wait is over. Hold the single rule above all — never block the loop — because it explains more of `hathor-core`'s design than any other principle. With Chapter 1's objects giving us things and this chapter giving us time, the next chapter assembles both into the recurring, named arrangements professionals reach for by reflex: the **design patterns**.

1. I/O-bound describes work whose speed is limited by input/output — network, disk, waiting on other systems — rather than by the processor. Most server software, including a node, is I/O-bound. ↩
2. CPU-bound describes work whose speed is limited by raw computation — the processor is the bottleneck. Examples: hashing, encryption, large numeric loops. ↩
3. A blocking call does not return control to your program until it has finished. While it waits, the calling thread can do nothing else. Its opposite is non-blocking (returns immediately, result delivered later). ↩

4. Concurrency is making progress on multiple tasks over the same period by interleaving them. It is about structure (dealing with many things at once), not necessarily simultaneity. ↩
5. Parallelism is executing multiple tasks at literally the same instant, which requires multiple processors/cores. All parallel programs are concurrent; not all concurrent programs are parallel. ↩
6. A thread is an independent sequence of execution within a process. Multiple threads in one process share the same memory, which makes communication easy and corruption easy. ↩
7. Preemptive multitasking lets the operating system suspend a thread at any moment to run another. The opposite is cooperative, where a task keeps running until it voluntarily yields. ↩
8. A race condition is a bug where the result depends on the unpredictable timing/order in which concurrent tasks run — e.g. two threads incrementing the same counter and losing an update. ↩
9. A lock (mutex) is a mechanism ensuring only one thread enters a critical section of code at a time, used to prevent race conditions. Misused locks cause deadlocks and slowdowns. ↩
10. A deadlock is a standstill where two or more threads each wait for a resource the other holds, so none can ever proceed. ↩
11. The Global Interpreter Lock is a mutex in standard CPython that allows only one thread to execute Python bytecode at a time. It simplifies the interpreter but prevents threads from running Python computation in parallel. ↩
12. Cooperative multitasking relies on each task voluntarily yielding control (e.g. at an `await`) so others can run. No task is forcibly interrupted, so a task that never yields blocks all others. ↩
13. A callback is a function passed to other code to be called ("called back") when a specific event happens or a result becomes available. ↩
14. A value is first-class if it can be stored in variables, passed as an argument, and returned from functions. In Python, functions are first-class values. ↩
15. A higher-order function is one that takes other functions as arguments and/or returns a function. `map`, `sorted(key=...)`, and decorators are examples. ↩
16. A future (a.k.a. promise) is an object representing a result that will be available later. You attach callbacks to it or `await` it. Twisted's version is the Deferred; JavaScript's is the Promise. ↩
17. A coroutine is a function that can suspend its execution (at an `await`) and later resume from exactly that point, preserving its local state. Defined with `async def` in Python. ↩
18. A thread pool is a managed set of reusable worker threads. Tasks are submitted to the pool rather than creating a new thread each time; used to run blocking or CPU-bound work off the main loop. ↩